

When Do Counterexamples Improve Formal Specifications? Failure-Mode and Substrate Dependence in Test-to-Spec Refinement

Adarsh Agrawal

Senior Member, IEEE

Abstract

Counterexample feedback is widely assumed to refine formal specifications, yet its effectiveness depends critically on matching feedback type to the dominant failure mode. We evaluate three counterexample delivery mechanisms—textual critique, machine-validated prompt, and enforced constraint—across 127 mined Lean tasks (Verina) and 34 curated formal tasks (FVSpec/VeriContest) spanning two substrates (Lean, Verus) and two regimes (binary selection, open synthesis).

Primary result: On 127 Verina specification-synthesis tasks, free-form textual critique significantly outperforms both a second draft (27.0% vs. 10.7% recovery on 122 initial failures, McNemar $p = 9 \times 10^{-5}$) and machine-validated exact counterexamples (27.0% vs. 18.9%, $p = 0.031$). Failure-mode analysis reveals why: 115 of 122 failures are type or decidability errors; free-form critique addresses these, while exact semantic witnesses rarely fire.

This inverts the 34-task curated result, where a controlled ablation shows the lift comes from re-authoring, not counterexamples. On 10-task Lean synthesis, a counterexample-free second draft reproduces the entire gain over one-way generation (provable-spec rate: second draft 64.2%, full loop 63.6%, one-way 57.8%). On 10-task grammar-constrained Lean selection, injecting a validated counterexample degrades a weaker model (46.7% vs. 96.7% one-way), while stronger models stay at 100%.

Contribution: Counterexample utility is substrate-, validator-, and task-distribution dependent. Practitioners should match feedback mechanism to observed failure mode: textual critique for syntactic/type repair, exact witnesses only when semantic failures dominate. We deliver decision-relevant evidence across 161 tasks, native proof checking (Lean 4.33, Verus 0.2026), and ablation-pinned mechanisms.

Introduction

Formal verification establishes that an implementation satisfies a specification. It does not establish that the specification captures the intended behavior. A weak contract may be provable while omitting an edge case, constraining only visible examples, or encoding the wrong relation. This gap has made counterexample-guided refinement an attractive pattern for LLM-based specification synthesis: produce a draft, obtain a distinguishing example, revise, and then invoke the native verifier.

Classical CEGIS and CEGAR make a counterexample a machine-readable obligation for the next hypothesis (Clarke et al., 2000; Alur et al., 2013). An API model instead usually

receives the counterexample as context. That difference matters, but it is not the only one. A semantic witness can refine a specification that already compiles and expresses the wrong behavior. It cannot directly repair a draft that fails the type checker, lacks a decidability instance, or does not parse. A free-form diagnostic may be better for those failures because it can explain why no executable semantic check is currently possible.

We study this distinction with TEST2SPEC2PROOF. The evaluation combines a natural open-synthesis benchmark with controlled mechanism experiments:

- **Natural-data evaluation.** On 127 mined Verina tasks, free-form textual diagnosis substantially outperforms a plain second draft. Its advantage is concentrated in type and decidability failures.
- **Controlled mechanism study.** On 34 curated FVSpec and VeriContest tasks, simpler prompting is sufficient on average, while the weaker model response exposes model-dependent sensitivity to witness injection.
- **Cross-substrate boundary.** A hard witness gate improves a small Lean subset but reverses on a near-saturated Verus subset. Enforcement is therefore a substrate-dependent intervention.

The resulting claim is deliberately conditional: feedback helps when its information matches the current failure mode. This reframing reconciles what would otherwise look like contradictory results across mined tasks, curated tasks, models, and formal substrates.

Related work

Counterexample and verifier-guided refinement. CEGAR and syntax-guided synthesis established the iterative pattern of checking a candidate and feeding a violated obligation back into search (Clarke et al., 2000; Alur et al., 2013). Recent LLM systems apply verifier or property feedback to verified-code generation and repair, including AlphaVerus, VerifyThisBench, property-oriented minimal feedback, ExVerus, and property-guided synthesis (Aggarwal et al., 2025; Deng et al., 2025; He et al., 2025; Yang et al., 2026; Pereira et al., 2026). These systems demonstrate that feedback can help, while our experiments ask which feedback representation helps for which failure mode and whether the gain remains after controlling for a second model call.

Specification synthesis and semantic fidelity. Property-based testing provides an important foundation for treating executions as partial behavioral evidence (Claessen and Hughes,

2000). Recent work studies formal specification inference, invariant generation, tool-assisted autoformalization, type-versus-semantic error stratification, traceable requirement refinement, and test-only specification synthesis (Le-Cong et al., 2025; Sun et al., 2025; Guo et al., 2025; Dai et al., 2026; Ye et al., 2026; Liu et al., 2026). This literature increasingly separates syntactic or native validity from semantic alignment. We add a matched comparison of textual diagnosis, validated witnesses, hard enforcement, and re-drafting.

Benchmarks for verifiable coding. New benchmarks now cover modular Lean generation, coding-interview proofs, curated end-to-end verification, large multi-language vericoding, aligned cross-substrate tasks, property-based Lean challenges, competitive-programming verification, and specification autoformalization (Ye et al., 2026; Dougherty and Mehta, 2025; Thakur et al., 2025; Bursuc et al., 2025; Zhao et al., 2026; Dougherty et al., 2026; Xie et al., 2026; Agarwal et al., 2026). They consistently expose different difficulty in code, specification, and proof generation. We use three of these task sources to study the feedback policy itself rather than propose another aggregate benchmark.

Method

Treatment definitions

The treatments differ only in what the model sees before revising its first draft. Table 1 separates showing a witness from enforcing it.

second draft asks the model to reconsider its answer without giving it new information. *textual diagnosis* explains what appears wrong. *validated witness* shows a checked input-output example but lets the model decide how to respond. *enforced witness* additionally rejects a revision that still mishandles that example. The model never sees the witness validator or held-out fidelity labels while writing the specification.

The four-method controlled study uses a related three-call protocol. *one-way* sees the task description; tests-to-spec additionally sees visible examples; spec-to-tests begins with a weak contract and requests a distinguishing behavior; and *bidirectional* infers from tests, requests a witness, validates it, and revises. Most validated witnesses in this study come from the evaluation search rather than the model, so this experiment measures revision given a valid witness rather than autonomous witness discovery.

Figure 1 summarizes the failure-aware policy evaluated by these treatments. Native validity and held-out semantic fidelity are checked separately before selecting a revision signal.

Tasks, substrates, and models

Table 2 summarizes the three evaluations. Verina (Ye et al., 2026) provides 127 mined Lean specification tasks and a native validator. The controlled cohort combines ten FVSpec Lean tasks (Dougherty et al., 2026) and 24 VeriContest Verus tasks (Xie et al., 2026). Every controlled selection task contains an observed-only and a full contract whose certificates pass the native checker, so the outcome isolates semantic selection

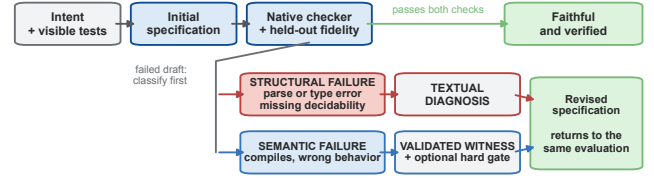


Figure 1: Failure-aware TEST2SPEC2PROOF workflow. A structural failure receives a textual diagnosis because an executable semantic witness cannot yet act on the draft. A specification that compiles but expresses the wrong behavior receives a validated witness, optionally backed by a hard gate. Every revision returns to the same native and held-out evaluation path. The routing policy is synthesized from the findings rather than evaluated as an integrated end-to-end system; Sections 4–6 evaluate its treatment arms independently.

rather than proof search.

The model portfolio includes Claude Sonnet 5, Claude Opus 5, Qwen3-Coder, and GPT-5.6 Terra where available. The Verina study uses Sonnet 5 and Qwen3-Coder. Because only two models are used there, we report model differences without claiming that they represent all model families.

Outcomes and failure modes

We call a result *faithful and verified* when it passes the native Lean or Verus checker and captures the intended behavior rather than only the visible examples. Native acceptance alone is not enough.

For open synthesis, we distinguish:

- **Type or decidability failure:** the specification does not compile as a decidable Lean predicate, so no semantic witness can yet be evaluated.
- **Semantic fidelity failure:** the specification compiles but fails a held-out intended-property check.

This distinction explains the main result. In Verina, 115 of 122 failed task-model cases are type or decidability failures and only seven are semantic fidelity failures.

Statistical design

We analyze results by task. For Verina, this gives 88 tasks with at least one initial failure across the two models. A task counts as recovered when at least one initially failed model succeeds after revision. Confidence intervals come from 10,000 resamples of whole tasks. Tests over individual model or seed runs are reported only as secondary diagnostics. For the controlled study, we first average model and seed results within each task and apply Holm correction within each family of comparisons.

Table 1: Feedback treatments. Showing a validated witness and enforcing it are different interventions.

Treatment	Revision information	Validated	Enforced	Calls
<i>second draft</i>	Bare request to reconsider and finalize	no	no	2
<i>textual diagnosis</i>	Free-form diagnostic or critique	no	no	3
<i>validated witness</i>	Oracle-validated input-output witness in the prompt	yes	no	3
<i>enforced witness</i>	Validated witness plus a pre-check gate	yes	yes	3 + gate

Table 2: Experimental design. We analyze results by task, keeping model and seed repetitions together.

Dataset	Tasks	Models	Seeds	Substrate	Primary role
Verina	127	2	1	Lean 4.33.1	Natural open synthesis
FVSpec	10	4	3–9	Lean 4.33.0	Selection and synthesis mechanism
VeriContest	24	4	3	Verus 0.2026.08.09	Cross-substrate selection

Natural-data result: textual diagnosis repairs the dominant failure

The Verina experiment begins from one shared initial synthesis for each task and model. The three revision arms are applied only to the 122 failed task-model cells: 34 Sonnet 5 failures and 88 Qwen3-Coder failures. At the task level, 88 of 127 tasks contain at least one initial failure.

Primary task-level comparison

textual diagnosis recovers 27 of 88 tasks (30.7%, 95% bootstrap CI [21.6, 40.9]%), compared with 12 of 88 (13.6%, [7.0, 21.6]%) for *second draft* and 20 of 88 (22.7%, [14.8, 31.8]%) for *validated witness*. The paired task-level comparison, using 10,000 whole-task resamples, favors textual diagnosis over a plain second draft by 17.1 pp ($p = 0.0013$); 19 tasks are discordant, with 15 favoring textual diagnosis and four favoring the second draft. The contrast with a validated witness is +8.0 pp ($p = 0.052$), with nine of 13 discordant tasks favoring textual diagnosis. Validated witness exceeds second draft by 9.1 pp ($p = 0.033$), with ten of 14 discordant tasks favoring the witness.

Textual diagnosis and validated witness each use three model calls, while the second-draft baseline uses two (Table 1). The textual-versus-validated comparison is therefore budget-matched; the textual-versus-second-draft comparison combines feedback content with one additional generation call.

The cell-level view supplies the same ordering but is not the primary inferential unit. Across 122 failed task-model cells, *textual diagnosis* recovers 33 (27.0%), *validated witness* recovers 23 (18.9%), and *second draft* recovers 13 (10.7%). The corresponding episode-level McNemar diagnostics are 9×10^{-5} for textual versus second draft, 0.031 for textual versus validated, and 0.041 for validated versus second draft.

Failure-mode interaction

The result is not evidence that free-form text is intrinsically superior to an executable witness. It is evidence that the Verina failure population is dominated by failures for which an executable witness cannot yet act. Among the 115 type or decidability failures, textual diagnosis recovers 33 (28.7%), validated witnesses recover 20 (17.4%), and a second draft

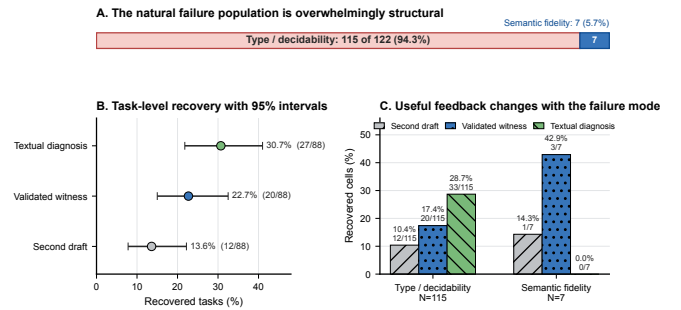


Figure 2: Failure-mode matching on Verina. (A) Type and decidability errors comprise 115 of 122 initial failed task-model cells. (B) Textual diagnosis has the highest task-level recovery among the 88 tasks with at least one initial failure. (C) The descriptive cell-level breakdown shows why: textual diagnosis leads on structural failures, while a validated witness is directionally strongest on semantic failures. The semantic stratum contains only seven cells and should not support a strong comparative conclusion.

recovers 12 (10.4%). Among the seven semantic failures, validated witnesses recover three (42.9%), the second draft recovers one (14.3%), and textual diagnosis recovers none. The latter stratum is too small for a strong comparative conclusion, but its direction is consistent with the failure-mode account.

Both Verina models show the same ordering. The effect is larger for Qwen3-Coder, which contributes the larger initial failure pool. Removing Qwen3-Coder leaves a directional but underpowered Sonnet-only comparison; removing Sonnet retains the significant Qwen-only comparison. We therefore interpret model as a moderator and avoid claiming that the effect is uniform across model families.

Controlled mechanism experiments

The 34-task study isolates the boundary

The controlled study contains 34 tasks, four methods, four models, and three seeds, for 1,632 completed episodes. Both candidate contracts pass their native checker, so proof acceptance is 100% by construction and cannot distinguish faithful from weak selections. The episode count reflects 34 independent tasks, with model and seed serving as within-task

repetitions; task-level intervals account for this clustering.

Tests-to-spec obtains the highest faithful-verified rate (58.6%), followed by *one-way* (53.9%), *bidirectional* (53.4%), and spec-to-tests (28.4%). At the task level, *bidirectional* differs from *one-way* by -0.5 pp with a whole-task interval of $[-3.9, +2.9]$ pp and an exact sign-test $p = 0.804$. Relative to tests-to-spec, *bidirectional* is lower by 5.1 pp, interval $[-7.8, -2.5]$ pp, sign-test $p = 0.00149$. Most witnesses are supplied by the evaluation search. This makes the study a clean test of how models revise when given a valid witness, but not a measure of how often models can discover such witnesses by themselves.

Grammar-constrained selection reveals model-dependent sensitivity

Structured output removes the original parsing problem. On ten FVSpec Lean tasks, four models, and three seeds, *one-way* reaches 99.2% faithful selection while *bidirectional* reaches 86.7%, a difference of -12.5 pp with task-cluster interval $[-18.3, -6.7]$ pp. The secondary paired diagnostic is $p = 0.0003$. The change is concentrated in Qwen3-Coder: its one-way rate is 96.7%, while the bidirectional rate is 46.7%. Sonnet 5, Opus 5, and Terra remain at or near 100% under every method.

The same selection pattern appears on 24 VeriContest tasks under native Verus. *one-way*, tests-to-spec, and spec-to-tests each reach 100%; *bidirectional* reaches 95.1%, a difference of -4.86 pp with interval $[-8.68, -1.74]$ pp. Fourteen discordant pairs favor one-way and none favor the loop. The cross-substrate result therefore supports a narrow conclusion: injecting an exact witness can destabilize a weaker model even when parsing is reliable.

Open synthesis shows that re-drafting is the active ingredient

The selection design is intentionally a binary mechanism probe. A separate Lean experiment requires each model to author a complete `FVspec.Spec` namespace and proof. Expanding from three to nine seeds improves within-task precision while retaining ten independent tasks.

Across 360 paired episodes, provable-spec rates are 57.8% for *one-way*, 63.6% for the full bidirectional loop, and 64.2% for a counterexample-free second draft. The full loop’s difference from one-way is $+5.83$ pp, but the task-cluster interval $[-1.7, +14.2]$ pp crosses zero. More importantly, *second draft* exceeds one-way by $+6.39$ pp with a task-cluster interval of $[+1.9, +10.6]$ pp, and is indistinguishable from the full loop: $+0.56$ pp for second draft minus loop, interval $[-7.2, +7.5]$ pp. The synthesis gain is therefore a second authoring-pass effect, not evidence that the counterexample itself improved the specification.

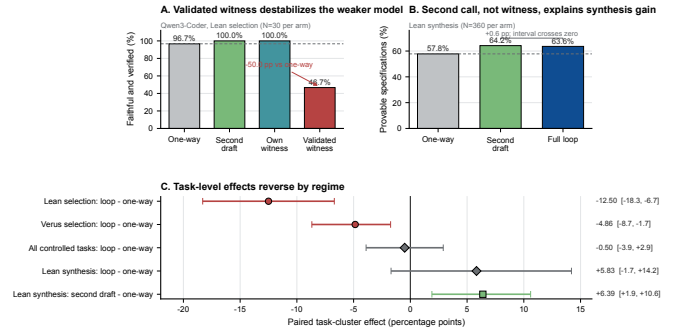


Figure 3: Controlled mechanism results. (A) On grammar-constrained Lean selection, Qwen3-Coder remains near ceiling after a bare second draft or its own unvalidated witness, but falls to 46.7% after injection of the validated witness. (B) In open Lean synthesis, a counterexample-free second draft and the full loop have nearly identical provable-spec rates. (C) Paired task-cluster effects show that the loop reverses across regimes and substrates, while the second-draft effect is the stable positive contrast.

Exploratory enforcement and substrate boundary

Hard enforcement is evaluated in an explicitly exploratory probe, separately from validated prompting. A revised specification must classify the witness correctly before the native checker is invoked. On three Lean fidelity tasks, *enforced witness* reaches 55.6% faithful success, compared with 33.3% for *second draft* and 38.9% for *textual diagnosis*. The contrast with the second draft is $+22.2$ pp, but the sample contains only 18 paired cells and is not significant ($p = 0.125$).

On nine near-saturated Verus tasks, *second draft* reaches 90.7%, *enforced witness* reaches 83.3%, and *textual diagnosis* reaches 77.8%. Enforcement is 7.4 pp below the second draft ($p = 0.39$). The enforcement gate and fidelity outcome also share a related mutant-kill signal, so this experiment measures mechanism behavior rather than fully independent label validation. The near-ceiling second-draft baseline limits the available headroom, so the reversal is interpreted as directional.

Together these results strengthen the conditional thesis: an exact witness is useful when the specification is semantically evaluable, the fidelity task has headroom, and the model can respond without abandoning a correct hypothesis.

Discussion

Why the results reverse. The natural and controlled studies expose different failure populations. Verina is dominated by type and decidability failures. Textual diagnosis can name the compiler-level obstacle and suggest a repair, while a semantic witness cannot execute. The curated candidate pools deliberately remove that obstacle by supplying native-valid candidates. In that setting, a second draft is often sufficient, and exact witness injection can add distraction without adding usable information.

What practitioners should measure. A deployment should first classify whether the current artifact fails to parse, type-

check, prove, or satisfy a held-out semantic obligation. Textual diagnosis is appropriate for structural failures. Validated examples are appropriate for semantic failures. A hard gate can prevent a known witness from being ignored, but it does not guarantee that the resulting specification generalizes beyond that witness.

Why proof acceptance is not enough. All 1,632 selections in this campaign pass a native checker because the candidate pool was constructed that way, while faithful success ranges from 28.4% to 58.6% by method. This is an evaluation design, not an estimate of how often real specifications are weak. It demonstrates why formal acceptance and intended-property fidelity must be reported separately.

Reproducibility. The anonymized supplement provides an offline verification path for the reported results from cached outcomes without making new model calls. It includes task-level results, prompts, output formats, analysis scripts, a data dictionary, and environment details.

Broader impact. Matching feedback to failure mode can make formal-specification workflows more reliable and reduce wasted model and verifier calls. The corresponding risk is false confidence: a mismatched or over-weighted witness can steer revision away from a correct hypothesis while the final artifact still passes a native checker. Native verification should therefore be paired with held-out fidelity checks and explicit reporting of the feedback policy.

Limitations

The Verina experiment uses two model families, so model is treated as a fixed moderator. The open-synthesis analysis has ten independent tasks; additional seeds improve within-task precision rather than enlarging the task population. The seven semantic-failure cells provide directional evidence for validated witnesses but do not support a precise comparative estimate. The enforcement experiments are exploratory probes, with three Lean tasks and a near-ceiling Verus subset.

Intent labels come from benchmark-authored reference behavior rather than independent human annotation. A three-model adjudication ensemble agrees with the controlled oracle on 31 of 34 tasks, but agreement is imperfect and does not replace human validation. The evaluation covers Lean and Verus benchmarks, not arbitrary repositories, policy languages, temporal systems, or ambiguous user requirements.

Conclusion

Counterexample feedback is not a single treatment with a universal effect. On 127 mined Lean tasks, free-form textual diagnosis performs best because 94.3% of failed task-model cells are type or decidability errors. On curated native-valid tasks, a plain second draft matches exact counterexample prompting, and responses to witness injection vary by model. In open Lean synthesis, the counterexample-free second draft accounts for the observed improvement. Machine enforcement

varies across the Lean and Verus probes.

The operational lesson is simple: diagnose the failure mode before choosing the feedback mechanism. Use textual diagnostics for parse, type, and decidability failures. Use executable witnesses for specifications that already compile but miss intended behavior. Enforce a witness when ignoring that witness is unacceptable, while still evaluating against independent held-out behavior. A proof establishes the chosen statement. The feedback policy helps determine whether that statement is the right one.

References

- Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *CAV*, 2000.
- Rajeev Alur, Rastislav Bodik, Garvit Juniwal, et al. Syntax-guided synthesis. In *FMCAD*, 2013.
- Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *ICFP*, 2000.
- Pranjal Aggarwal, Bryan Parno, and Sean Welleck. AlphaVerus: Bootstrapping formally verified code generation through self-improving translation and tree refinement. In *ICLR*, 2025.
- Thanh Le-Cong, Bach Le, and Toby Murray. Can LLMs reason about program semantics? A comprehensive evaluation of LLMs on formal specification inference. In *ACL*, 2025.
- Chuyue Sun, Viraj Agashe, Saikat Chakraborty, et al. ClassInvGen: Class invariant synthesis using large language models. arXiv:2502.18917, 2025.
- Quinn Dougherty and Ronak Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *LLM4Code at ICSE*, 2025.
- Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking verifiable code generation. In *ICLR*, 2026.
- Amitayush Thakur, Jasper Lee, George Tsoukalas, et al. CLEVER: A curated benchmark for formally verified code generation. arXiv:2505.13938, 2025.
- Xun Deng, Sicheng Zhong, Baris Bayazit, et al. VerifyThis-Bench: Generating code, specifications, and proofs all at once. arXiv:2505.19271, 2025.
- Lehan He, Zeren Chen, Zhe Zhang, Xiang Gao, and Lu Sheng. Effective LLM code refinement via property-oriented and structurally minimal feedback. arXiv:2506.18315, 2025.
- Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, et al. A benchmark for vericoding: Formally verified program synthesis. arXiv:2509.22908, 2025.
- Qi Guo et al. Autoformalizer with tool feedback. arXiv:2510.06857, 2025.

- Haoyu Zhao, Ziran Yang, Jiawei Li, et al. AlgoVeri: An aligned benchmark for verified code generation on classical algorithms. In *ICML*, 2026.
- Jun Yang, Yuechun Sun, Yi Wu, et al. ExVerus: Verus proof repair via counterexample reasoning. arXiv:2603.25810, 2026.
- Zhe Ye, Aidan Z. H. Yang, Huangyuan Su, et al. Intent-aligned formal specification synthesis via traceable refinement. arXiv:2604.10392, 2026.
- Andre G. Pereira, Augusto B. Correa, and Jendrik Seipp. Property-guided LLM program synthesis for planning. arXiv:2605.16142, 2026.
- Anmol Agarwal, Natalie Neamtu, Pranjal Aggarwal, et al. Verus-SpecGym: An agentic environment for evaluating specification autoformalization. arXiv:2605.26457, 2026.
- Quinn Dougherty, Max von Hippel, Simon Henniger, Hazel Shackleton, and Mike Dodds. FVSpec: Real-world property-based tests as Lean challenges. arXiv:2606.01008, 2026.
- Zichen Xie, Mrigank Pawagi, Yuxin Liu, et al. VeriContest: A competitive-programming benchmark for verifiable code generation. arXiv:2605.08553, 2026.
- Chengxiao Dai, Zhaokun Yan, and Zhanhui Lin. The signal-coverage matrix: Stratifying type and semantic errors in statement autoformalization. arXiv:2606.28013, 2026.
- Tianhai Liu, Maximilian Müller, Tobias Hey, Vitus Lüntzel, et al. Can formal specifications be synthesized from tests alone? In *ASE*, 2026.

Supplementary experimental detail

Primary result summary

Additional checks

Both Verina models show the same treatment ordering. The textual-diagnosis advantage is larger for Qwen3-Coder, which also contributes more initial failures, but removing either model preserves the direction of the result. Most validated witnesses in the controlled study are supplied by the evaluation search. We therefore interpret that experiment as a study of revision given a valid witness.

The natural-data comparison, controlled four-method comparison, and one-way versus bidirectional synthesis comparison were planned before their respective evaluations. Failure-mode and model-specific analyses are used to explain the observed pattern. The planned criterion for a large reduction in accepted weak contracts was not reached, so we do not claim that the full loop is generally better than simpler prompting.

FVSpec and VeriContest fidelity labels come from their benchmark reference behavior. An independent model-based check agreed with the benchmark label on 31 of 34 controlled tasks. We treat these labels as benchmark intent, not as a substitute for human intent.

Reproducibility and availability

Supplement contents. The anonymized OpenReview supplement contains task-level CSV outcomes, cached parsed results, failure-mode labels, recovery flags, prompt templates, JSON output schemas, analysis and figure scripts, a data dictionary, and environment files. Its root directory is `test2spec2proof_supplement`. The included manifest maps reported artifacts to the corresponding analysis scripts. Raw provider response bodies and identifying request metadata are excluded. The cached outcomes support offline verification of the reported aggregate results and figures.

After installing the pinned dependencies with `pip install -r analysis/requirements.txt`, running `make reproduce` writes all reproduced artifacts to `results/`. The replay takes less than 30 seconds on a laptop and, after dependency installation, requires no network access, model API calls, or GPU.

Hosted-model settings. Table 4 gives the exact endpoints and evaluation periods. Generations use a maximum output length of 4,096 tokens. Temperature 0.6 is used where supported; the Claude 5 endpoints omit temperature and use their configured reasoning mode. Anthropic does not expose a seed parameter, so its replicate indices denote independent stochastic calls. Qwen3-Coder and Terra receive the per-episode seed where supported, with provider-level best-effort determinism. Qwen3-Coder uses `enable_thinking=false`; the other models use their standard reasoning mode. No additional system message is used beyond the released task templates.

Transient rate-limit and server failures are retried up to three times with 1, 2, and 4 second waits. The first successful response is analyzed. Permanent provider failures are recorded as failed episodes. Each episode comprises one or more model calls, depending on whether the treatment adds diagnosis, witness generation, or revision. This produces approximately 12,700 canonical calls across 4,244 episodes, or about three calls per episode on average; retry attempts are excluded. No model training or fine-tuning is performed.

Episode and compute accounting. The Verina campaign contains 254 initial generations. Its 122 failed task-model cells each enter three revision arms, producing 366 revision episodes and 620 episodes in total. The controlled 34-task selection campaign contains 1,632 episodes; the 480-episode grammar-constrained FVSpec study is a separate rerun. The nine-repeat synthesis study contains 1,080 episodes and the enforcement study contains 432, for approximately 4,244 reported episodes.

All orchestration, native checking, and analysis run on an AWS EC2 `c6i.2xlarge` worker with eight Intel Xeon Ice Lake vCPUs, 16 GB RAM, 100 GB gp3 storage, and Ubuntu 22.04. No GPU is used. Table 5 reports approximate wall-clock time.

The reported campaigns use approximately 38 million input tokens, 9.2 million output tokens, and \$340 in API charges. Including preliminary and development runs, the full project used approximately 72 hours of wall-clock time and \$1,200 in API charges.

Table 3: Primary outcomes. Task-level intervals are used whenever repeated model or seed observations share a task.

Experiment	Comparison	Effect	Interval	Result
Verina task recovery	Textual vs second draft	+17.1 pp	task paired	$p = 0.0013$
Verina task recovery	Textual vs validated CE	+8.0 pp	task paired	$p = 0.052$
Controlled selection	Bidirectional vs one-way	-0.5 pp	$[-3.9, +2.9]$	$p = 0.804$
Lean selection	Bidirectional vs one-way	-12.5 pp	$[-18.3, -6.7]$	secondary $p = 0.0003$
Verus selection	Bidirectional vs one-way	-4.86 pp	$[-8.68, -1.74]$	secondary $p = 0.0001$
Lean synthesis	Second draft vs one-way	+6.39 pp	$[+1.9, +10.6]$	secondary $p = 0.014$
Lean synthesis	Loop vs second draft	-0.56 pp	$[-7.5, +7.2]$	secondary $p = 0.90$

Table 4: Hosted-model configuration. Dates are in 2026.

Model	API identifier	Provider	Evaluation dates
Sonnet 5	claude-sonnet-5-20260514	Anthropic	Jul. 22–Aug. 18
Opus 5	claude-opus-5-20260514	Anthropic	Jul. 22–Aug. 4
Qwen3-Coder	qwen3-coder-2025-0730	DashScope	Jul. 22–Aug. 18
Terra	gpt-5.6-terra-2026-05-14	OpenAI	Jul. 22–Aug. 4

Assets and licenses. Table 6 records the benchmark and verifier versions. Exact source revisions are included in the supplement. The submission does not redistribute benchmark data, source code, or model weights; it provides task identifiers and derived outcomes.

The new analysis and reproduction scripts are released under the MIT License. Derived outcome tables are released under CC-BY-4.0. Separate license files and a README map each license to its corresponding files. The README also documents every CSV column, the prompt and schema directories, and the exact reproduction command.

Table 5: Compute for the reported campaigns. The grammar-constrained cohort is separate from the 34-task controlled-selection campaign.

Campaign	Episodes	Wall-clock time
Verina initial and revision	620	6.0 h
Controlled selection	1,632	4.0 h
Grammar-constrained FVSpec rerun	480	1.5 h
Nine-repeat Lean synthesis	1,080	5.0 h
Witness enforcement	432	2.0 h
Total reported campaigns	4,244	18.5 h

Table 6: Versions and licenses for existing assets.

Asset	Version used	License
Verina	July 2026 snapshot, 127 tasks	MIT
FVSpec	v0.1, 10 tasks	Apache 2.0
VeriContest	v1.0, 24 tasks	CC-BY-4.0
Lean	4.33.0 and 4.33.1	Apache 2.0
Mathlib	July 2026, Lean 4.33.x compatible	Apache 2.0
Verus	0.2026.08.09	MIT
Z3	4.13.0	MIT